

# AVALIAÇÃO 1 - PRÁTICA EXTENSIONISTA IV

Modelagem do Sistema de Informação  
Barbearia - aplicação no estilo PWA

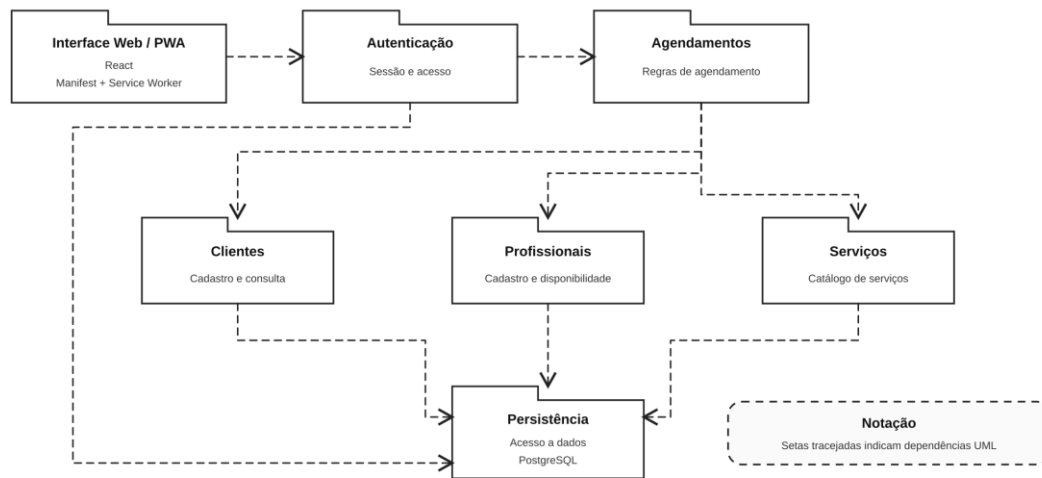
**Integrante:** Vanderlei Junior, Pedro Vier, Claudio Roberto da Silva, Leonardo Baumgarten Bauer e Gabriel Thomaz Fagundes.

**GitHub:** <https://github.com/PedroLVier/praticas-extensionistas-iv>

Este documento contém os diagramas solicitados e a descrição da infraestrutura escolhida para o projeto.

# 1. Diagrama UML de Pacotes

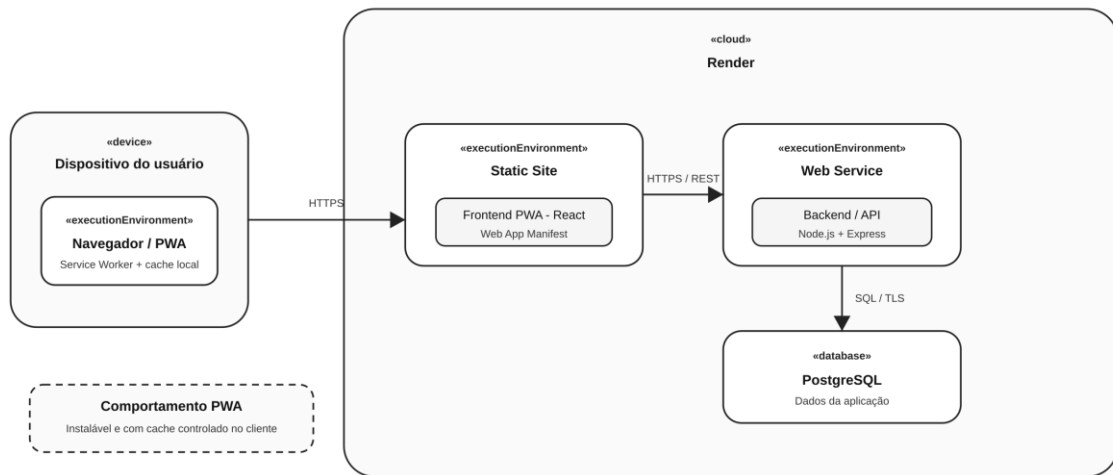
Diagrama UML de Pacotes - Barbearia



O diagrama mostra os principais pacotes da aplicação e as dependências entre eles. A interface Web/PWA se relaciona com autenticação e agendamentos. O módulo de agendamentos utiliza os dados de clientes, profissionais e serviços, enquanto a persistência concentra o acesso ao banco de dados PostgreSQL.

## 2. Diagrama de Arquitetura de Implantação

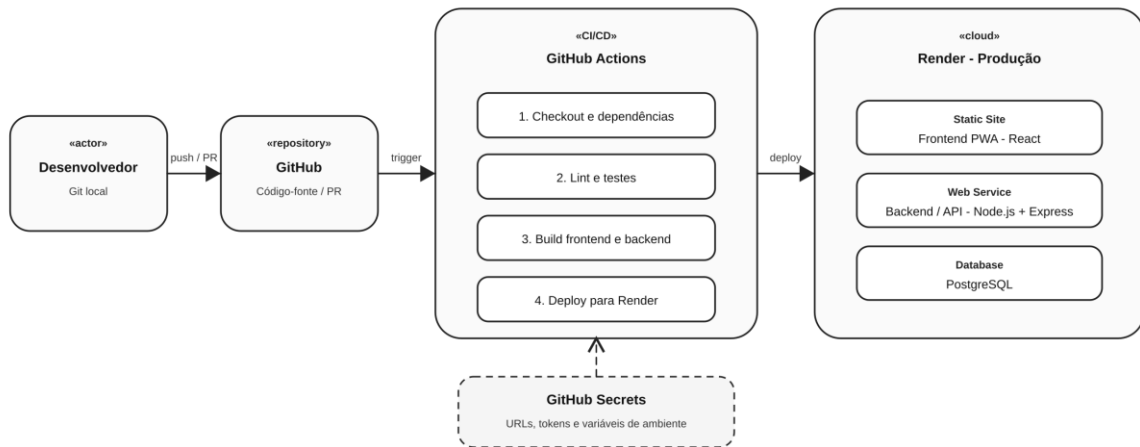
Diagrama de Arquitetura de Implantação - Barbearia



A aplicação será acessada pelo navegador do usuário. O frontend React/PWA ficará publicado no Render e fará comunicação com a API Node.js/Express por HTTPS/REST. A API será responsável por acessar o banco de dados PostgreSQL. A parte PWA inclui Manifest, Service Worker e cache no cliente.

### 3. Diagrama de Arquitetura DevOps

Diagrama de Arquitetura DevOps - Barbearia



Fluxo contínuo: versionamento → validação → build → publicação em produção

O código será enviado ao GitHub pelos desenvolvedores. O GitHub Actions executará as etapas de validação, testes e build antes da publicação. O deploy será realizado no Render, onde ficarão o frontend, o backend e o banco de dados. Informações sensíveis poderão ser armazenadas em GitHub Secrets e variáveis de ambiente.

## 4. Infraestrutura de Deploy e Publicação

A solução será publicada no Render. O frontend será uma aplicação React no formato PWA, o backend utilizará Node.js com Express e o banco de dados será PostgreSQL.

### Estrutura prevista

- Frontend PWA publicado como Static Site no Render;
- Backend / API publicado como Web Service no Render;
- Banco de dados PostgreSQL;
- Código e documentação armazenados no GitHub;
- GitHub Actions para testes, build e publicação;
- GitHub Secrets e variáveis de ambiente para informações sensíveis;
- Comunicação externa utilizando HTTPS.

### Justificativa da escolha

O Render foi escolhido porque permite concentrar os principais componentes do projeto em uma única plataforma e possui integração com repositórios Git. Para um projeto acadêmico de pequeno porte, isso reduz a quantidade de configurações necessárias para colocar a aplicação no ar.

A separação entre frontend, backend e banco de dados também facilita a organização da aplicação. O frontend e o backend se comunicam por HTTPS/REST e o banco de dados é acessado somente pela API.

### Comparação com outras alternativas

AWS, Microsoft Azure e Google Cloud oferecem mais serviços, opções de configuração e escalabilidade. Porém, para o escopo atual do trabalho, exigiriam uma estrutura maior de configuração e administração. O Render foi escolhido por ser mais simples para esta etapa e por atender frontend, backend e banco de dados.

### Configuração posterior

Para a implantação posterior, deverão ser criados no Render o Static Site do frontend, o Web Service do backend e o banco PostgreSQL. O repositório do GitHub deverá ser conectado à plataforma e as variáveis de ambiente necessárias deverão ser configuradas.